AI Project

Personal Trainer

Intelligent systems Engineering

Submitted to :
Dr. Samar M. Nour
Eng. Karim Mokhtar

FACULTY OF ENGINEERING

PROJECT REPORT

AI PERSONAL TRAINER

A Real-Time Pose-Estimation-Based Fitness Coaching System

Name	Library
Mohamed Ali Ragab	921230123
Mohamed Ashraf Mohamed	921230113
Mohamed Osman Khalifa	921230122
Omar Ahmed Hasan	921230089
Retal Mohamed Mohamed	921230045

Submission Date : May 2026

Academic Year 2025 – 2026

Abstract

The **AI Personal Trainer** is a real-time fitness coaching system designed to help users exercise safely and correctly using just a standard webcam. While professional personal trainers are expensive and standard workout apps only offer static video guides, this project uses computer vision to deliver personalized, real-time feedback on ordinary computers without requiring any extra hardware or internet connectivity.

The system uses Google's **MediaPipe Pose** framework to track 33 key body landmarks in real time. By calculating joint angles from these points, the system recognizes exercises and evaluates the user's form. Currently supporting **push-ups and squats**, the application uses a rule-based finite-state machine (FSM) built on exercise science guidelines to count repetitions and detect movement phases. The user receives instant, on-screen corrective feedback (like *"Go Lower"* or *"Good Rep"*), alongside calorie estimation based on the Metabolic Equivalent of Task (MET) formula.

To provide a complete user experience, the system features a **Tkinter** graphical interface, an **SQLite database** to store workout history, and a **Matplotlib dashboard** to display fitness trends over time. Testing shows that this lightweight approach is highly successful, achieving a **repetition counting accuracy of over 95%**. Ultimately, this project demonstrates a highly accurate, accessible, and modular solution that is easy to expand with more exercises or deep learning models in the future.

Table of Contents

1. Introduction

Physical inactivity is widely recognised as one of the leading risk factors for non-communicable diseases, including cardiovascular disease, type-2 diabetes, and musculoskeletal disorders. Despite growing awareness of the health benefits associated with regular exercise, adherence to structured fitness programmes remains low across populations worldwide. A significant contributor to this gap is the cost and inaccessibility of personalised coaching: professional personal trainers require ongoing financial commitment that is prohibitive for many individuals, while generic training applications provide static guidance that cannot adapt to a user's real-time movement quality.

Recent advances in computer vision — particularly the availability of efficient, accurate human pose estimation models — present an opportunity to bridge this gap. By processing video frames captured through a standard webcam, it is now feasible to extract precise skeletal keypoint data and derive biomechanically meaningful features in real time, on commodity hardware, without specialised depth cameras or wearable sensors. This capability forms the technological foundation of the AI Personal Trainer system described in this report.

The AI Personal Trainer integrates Google's MediaPipe Pose framework with a rule-based exercise analysis engine, a persistent data layer backed by SQLite, and a graphical user interface built in Tkinter to deliver a complete, end-to-end fitness coaching experience. The system is designed to be accessible, extensible, and deployable on any modern personal computer equipped with a webcam, requiring no internet connectivity during operation.

This report presents the full design, architecture, implementation, and evaluation of the system. Section 2 articulates the problem statement, Section 3 defines the project objectives, and Sections 4 through 11 describe the system overview, architecture, technologies, methodology, database design, and user interface in detail. Sections 12 through 14 critically evaluate the system's limitations, propose future enhancements — including a transition to supervised machine learning — and conclude the work.

2. Problem Statement

Access to high-quality, individualised fitness coaching is constrained by three interrelated barriers: cost, availability, and scalability. Professional personal trainers command significant per-session fees, making sustained engagement economically unviable for large segments of the population. Gym-based automated fitness equipment offers limited feedback, is confined to specific locations, and does not provide real-time corrective guidance. Consumer fitness applications, while widely used, deliver pre-recorded video content or generic plans that are entirely decoupled from the user's actual movement execution.

Exercising with poor form not only diminishes the effectiveness of training but also introduces a significant risk of acute and chronic injury. Without real-time feedback from a qualified coach, users commonly develop compensatory movement patterns that go undetected until injury occurs. This problem is particularly acute for beginners who lack proprioceptive awareness and the technical knowledge required to self-correct.

Existing computer-vision-based exercise analysis systems are either proprietary, require expensive hardware (e.g., depth cameras such as Microsoft Kinect), or are limited to research laboratory environments and are not available for consumer deployment. There is therefore a clear and unmet need for an accessible, hardware-agnostic, real-time exercise coaching system that can provide personalised feedback comparable in quality to a human trainer, using only a commodity webcam and a personal computer.

This project addresses that need by designing and implementing a complete software system that combines state-of-the-art pose estimation, biomechanical analysis, and user-centred interface design to provide effective, real-time coaching for common resistance exercises.

3. Objectives

The project is guided by the following primary and secondary objectives:

3.1 Primary Objectives

- To develop a real-time human pose estimation pipeline capable of detecting and tracking 33 body landmarks at a frame rate sufficient for live coaching feedback (target: ≥ 20 fps on mid-range hardware).
- To implement a biomechanically grounded, rule-based exercise recognition engine capable of accurately counting repetitions and assessing form quality for push-up and squat exercises.
- To design and implement a real-time corrective feedback mechanism that communicates actionable guidance to the user via on-screen overlays synchronised with each repetition cycle.
- To build a persistent data management layer using SQLite that stores user profiles, session logs, and aggregated statistics for longitudinal performance tracking.
- To create an intuitive graphical user interface using Tkinter that enables non-technical users to operate the system without prior training.

3.2 Secondary Objectives

- To implement a calorie estimation algorithm based on the MET model that utilises user-specific anthropometric data for personalised energy expenditure reporting.
- To generate analytical visualisations using Matplotlib that communicate historical training trends and motivate continued engagement.
- To design the system architecture with extensibility in mind, ensuring that future integration of supervised machine learning models requires only modular component replacement.
- To document the system thoroughly to meet academic submission standards and to serve as a basis for future research and development.

4. System Overview

The AI Personal Trainer is a single-machine, offline-capable desktop application written in Python. The system is structured around four tightly integrated functional layers that together realise the end-to-end coaching pipeline: the Vision Layer, the Analysis Layer, the Data Layer, and the Presentation Layer.

4.1 Vision Layer

The Vision Layer is responsible for acquiring raw video frames from the system webcam via OpenCV and passing each frame to the MediaPipe Pose inference pipeline. The output is a set of 33 three-dimensional normalised landmark coordinates, each associated with a visibility confidence score. This layer also handles frame pre-processing (colour space conversion from BGR to RGB as required by MediaPipe) and post-processing (overlay rendering of the skeleton and angle annotations onto the original frame).

4.2 Analysis Layer

The Analysis Layer receives the landmark set from the Vision Layer and implements the core intelligent functionality of the system. It computes joint angles from triplets of anatomical landmarks using vector mathematics, applies threshold-based state transition logic to recognise exercise phases and count repetitions, generates natural-language feedback strings based on the current movement state, and estimates caloric expenditure using the MET model.

4.3 Data Layer

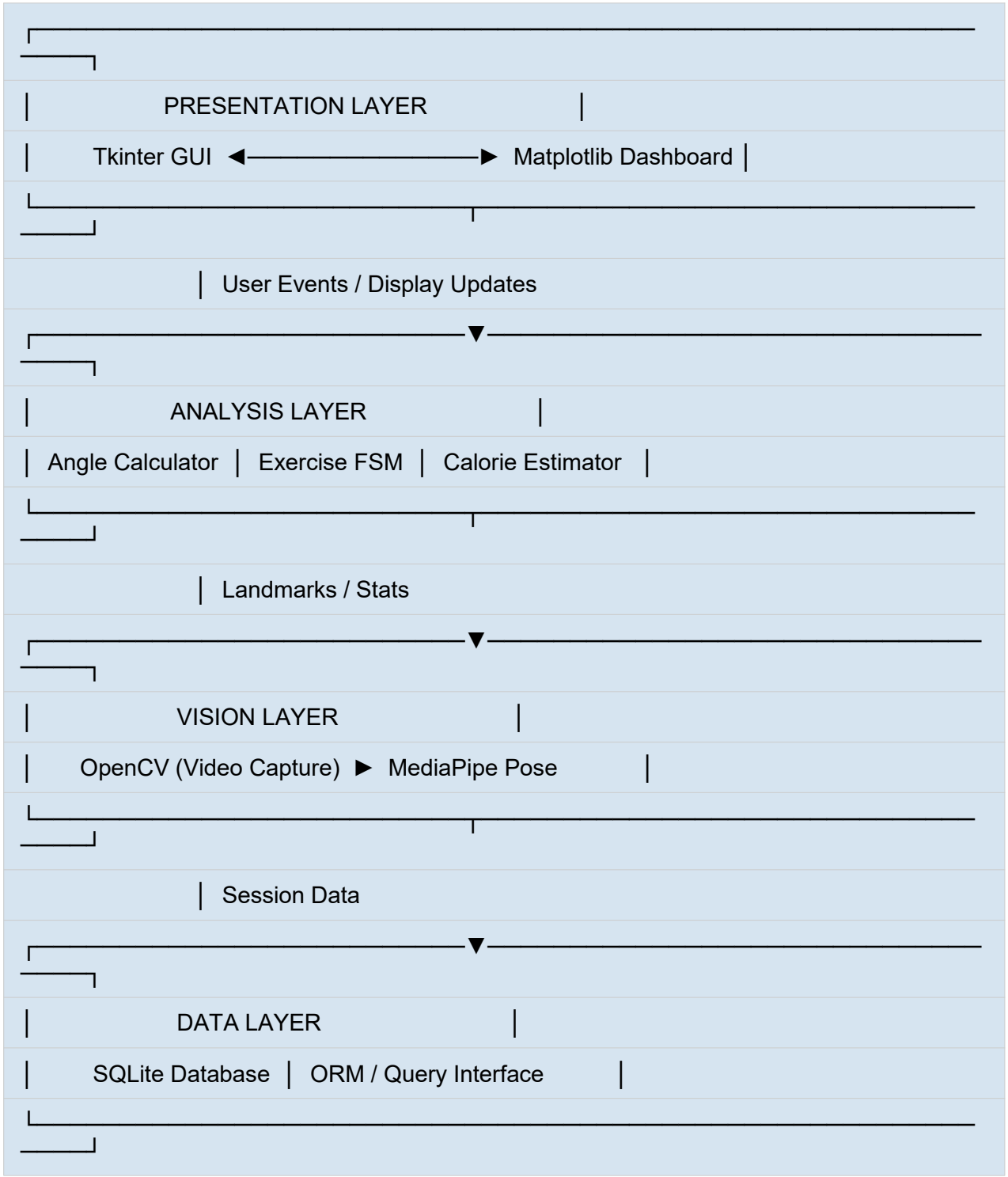
The Data Layer provides persistent storage and retrieval services for all user and session data. It is built on SQLite, a lightweight, serverless relational database engine embedded within Python's standard library. The Data Layer exposes a clean API to the Analysis and Presentation layers, abstracting raw SQL operations behind domain-specific methods (e.g., create_user, log_session, fetch_weekly_stats).

4.4 Presentation Layer

The Presentation Layer comprises the Tkinter GUI and the Matplotlib analytics dashboard. It provides the user-facing components through which users enter personal data, start and stop training sessions, monitor live workout statistics, and review historical performance data. The live webcam feed, annotated with skeleton overlays and real-time feedback, is embedded directly within the Tkinter window using the PIL/Pillow image conversion bridge.

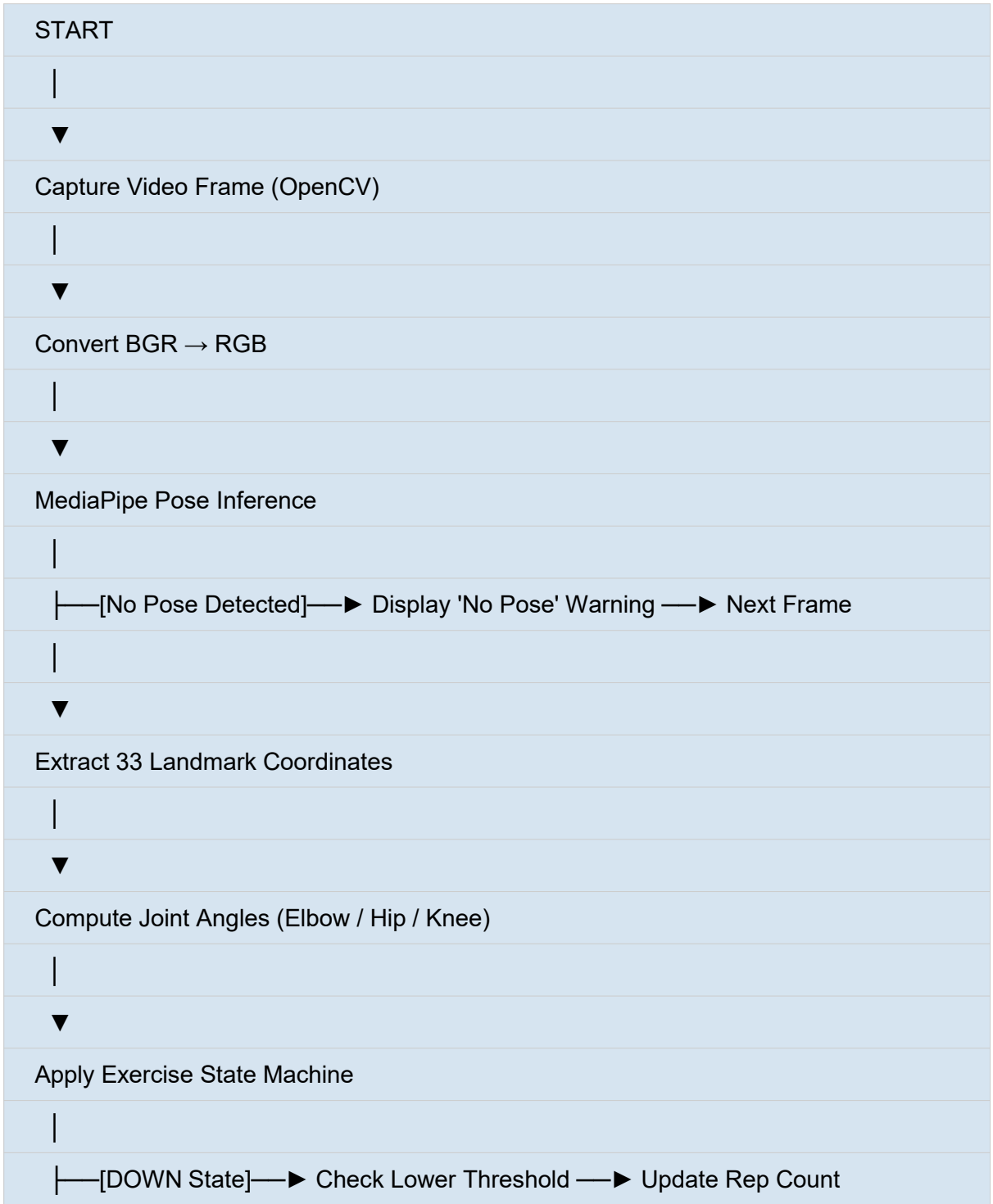
5. System Architecture

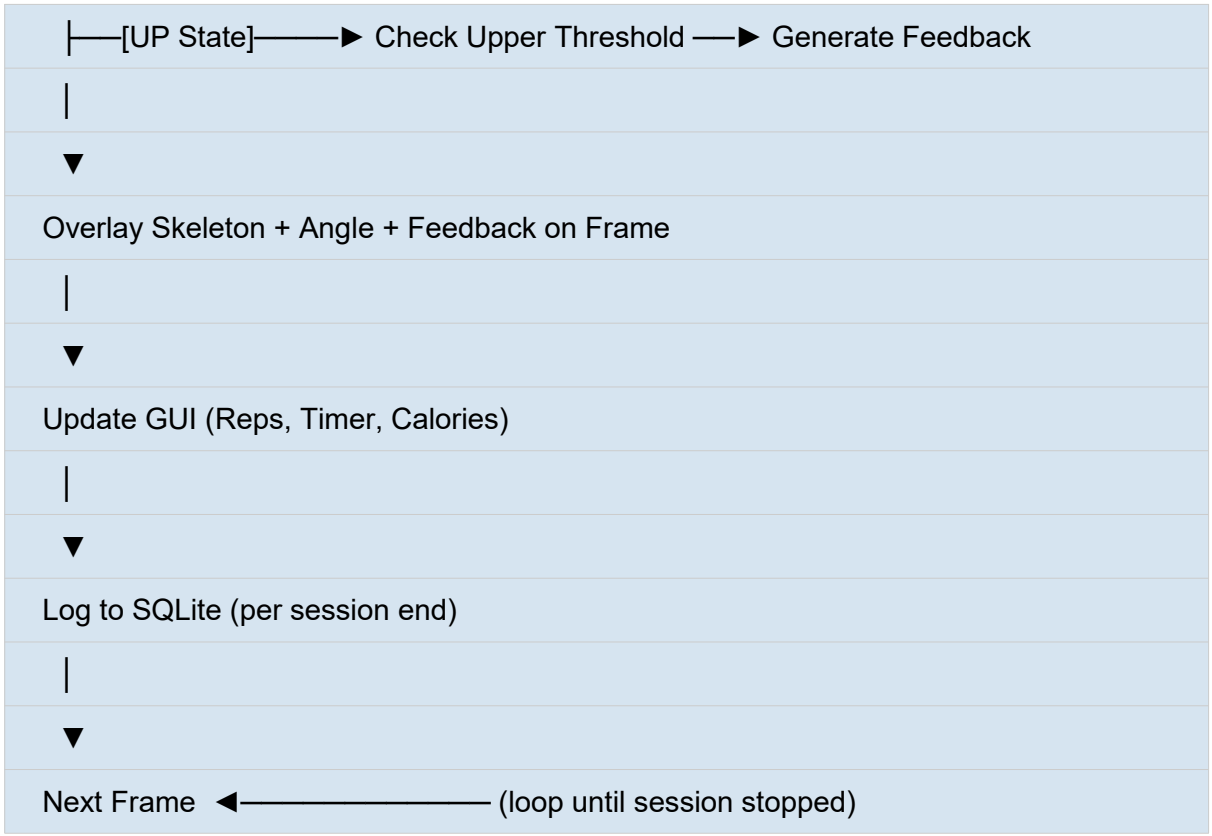
The system follows a layered, modular architecture designed for clarity of responsibility, ease of testing, and long-term maintainability. Figure 1 provides a conceptual representation of the major architectural components and their data flow relationships.



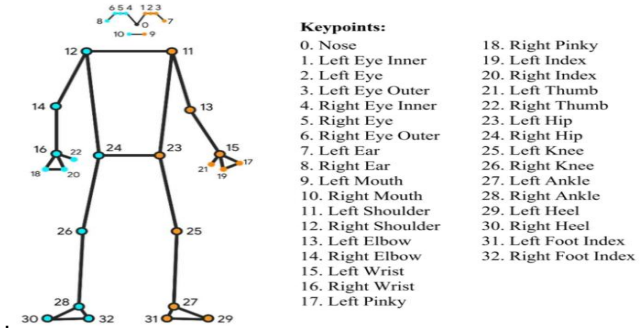
The architecture enforces unidirectional data flow between layers. The Vision Layer feeds landmark data upward to the Analysis Layer, which in turn supplies processed statistics to both the Presentation and Data layers. User interactions from the Presentation Layer trigger control signals (e.g., session start/stop) that flow downward to the Vision and Analysis layers. This separation of concerns ensures that any individual component can be upgraded or replaced — most critically, the rule-based Analysis Layer can be replaced with a trained machine learning model — without requiring changes elsewhere in the system.

5.1 Processing Pipeline Flowchart




├──[UP State]────► Check Upper Threshold ──► Generate Feedback
▼
Overlay Skeleton + Angle + Feedback on Frame
▼
Update GUI (Reps, Timer, Calories)
▼
Log to SQLite (per session end)
▼
Next Frame ◄──────────────── (loop until session stopped)

6. Technologies Used

Technology / Library	Role & Justification
Python 3.10+	Primary programming language. Chosen for its extensive scientific ecosystem, cross-platform compatibility, and widespread adoption in computer vision and machine learning research.
MediaPipe Pose	The **MediaPipe** Pose Landmarker task lets you detect landmarks of human bodies in an image or video. You can use this task to identify key body locations, analyze posture, and categorize movements. This task uses machine learning (ML) models that work with single images or video. The task outputs body pose landmarks in image coordinates and in 3-dimensional world coordinates.  **Keypoints:** 0. Nose 1. Left Eye Inner 2. Left Eye 3. Left Eye Outer 4. Right Eye Inner 5. Right Eye 6. Right Eye Outer 7. Left Ear 8. Right Ear 9. Left Mouth 10. Right Mouth 11. Left Shoulder 12. Right Shoulder 13. Left Elbow 14. Right Elbow 15. Left Wrist 16. Right Wrist 17. Left Pinky 18. Right Pinky 19. Left Index 20. Right Index 21. Left Thumb 22. Right Thumb 23. Left Hip 24. Right Hip 25. Left Knee 26. Right Knee 27. Left Ankle 28. Right Ankle 29. Left Heel 30. Right Heel 31. Left Foot Index 32. Right Foot Index
OpenCV (cv2)	Video capture, frame acquisition, colour space conversion, and annotation rendering. Industry standard for real-time computer vision applications.
Tkinter	Native Python GUI toolkit for the main application interface. Requires no external installation and integrates seamlessly with Python's standard library.
SQLite3	Serverless, embedded relational database for local data persistence. Zero-configuration deployment, ACID-compliant, and included in Python's standard library.
Matplotlib	Matplotlib: Visualization with Python Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.
Numerical Python(NumPy)	Numerical computing library used for vector arithmetic in joint angle calculation. Provides efficient array operations critical for real-time performance.
Pillow (PIL)	Image processing library used to convert OpenCV NumPy frames to Tkinter-compatible PhotoImage objects for GUI embedding.
Math (stdlib)	Built-in Python module used for trigonometric functions (atan2) in angle computation.

7. Methodology

This section provides a detailed technical description of the core algorithmic and engineering methods employed in the system. Each subsection addresses a distinct functional component of the Analysis Layer.

7.1 Pose Detection

Human pose estimation is performed using MediaPipe Pose, a machine learning solution developed and maintained by Google Research. The underlying model is a BlazePose architecture — a lightweight, efficient convolutional neural network specifically optimised for on-device human pose estimation. BlazePose operates in two stages: a detection stage that localises the person within the frame using a fast, single-shot detector, and a landmark regression stage that predicts 33 three-dimensional keypoints relative to the detected bounding region.

Each keypoint (landmark) provides normalised x and y coordinates in the range [0, 1] relative to the frame dimensions, a z coordinate representing depth relative to the hip midpoint (negative values indicate closer to the camera), and a visibility score in the range [0, 1] indicating the confidence that the landmark is within the frame and unoccluded. The 33 landmarks include key anatomical points such as shoulders, elbows, wrists, hips, knees, and ankles — all of which are necessary for analysing upper-body and lower-body exercises.

The system initialises MediaPipe Pose with the following configuration parameters:

- min_detection_confidence = 0.7: minimum confidence for the initial person detection step.
- min_tracking_confidence = 0.7: minimum confidence for the landmark tracking step in subsequent frames.
- model_complexity = 1: medium complexity model balancing accuracy and computational efficiency.

Frames for which MediaPipe returns no pose detection, or for which critical landmark visibility scores fall below the configured threshold, are flagged as invalid, and the user is notified via an on-screen warning. This prevents erroneous angle calculations and spurious feedback from being presented to the user.

7.2 Joint Angle Calculation

The primary feature used for exercise recognition is the joint angle — the angle formed at a particular joint by the two limb segments meeting at that point. Given three landmark points A (proximal segment), B (joint), and C (distal segment), the joint angle θ is computed using the arctangent of the cross product and dot product of the two limb vectors:

Vector BA = A - B (proximal limb segment, in 2D: [x_A - x_B, y_A - y_B])

Vector BC = C - B (distal limb segment, in 2D: [x_C - x_B, y_C - y_B])

$$\theta = |\text{atan2}(\text{cross}(BA, BC), \text{dot}(BA, BC))| \times (180 / \pi)$$

where: $\text{cross}(u,v) = u.x \times v.y - u.y \times v.x$

$\text{dot}(u,v) = u.x \times v.x + u.y \times v.y$

The use of atan2 rather than the simple inverse-cosine formulation provides two advantages: it correctly handles all four quadrants, eliminating sign ambiguity, and it avoids the numerical instability of acos near 0° and 180° where the function's derivative is steep. The result is converted from radians to degrees and its absolute value taken, yielding an angle in [0°, 180°] regardless of the relative orientation of the limb segments in the image plane.

The landmark triplets used for each tracked joint are defined as follows:

Joint	Landmark Triplet (A – B – C)
Right Elbow Angle	Right Shoulder – Right Elbow – Right Wrist
Left Elbow Angle	Left Shoulder – Left Elbow – Left Wrist
Right Knee Angle	Right Hip – Right Knee – Right Ankle
Left Knee Angle	Left Hip – Left Knee – Left Ankle
Hip Angle (torso inclination)	Right Shoulder – Right Hip – Right Knee

7.3 Exercise Recognition Logic

Exercise recognition and repetition counting are implemented as a deterministic finite-state machine (FSM) operating on the computed joint angles. Each exercise has two states — UP (rest/extended position) and DOWN (contracted/flexed position) — and a set of transition conditions defined by biomechanically validated angle thresholds.

7.3.1 Push-Up Detection

The push-up is modelled using the average elbow angle (mean of right and left elbow angles). The state transitions are as follows:

- UP state: elbow angle > 160°. The arms are fully or nearly extended. Feedback: 'Arms Extended — Ready'.
- Transition to DOWN: elbow angle < 90°. The chest has lowered sufficiently close to the floor. The system increments the pending-rep counter.
- Transition back to UP: elbow angle > 160°. The rep is confirmed and the repetition counter is incremented. Feedback: 'Good Rep'.
- Partial descent (90° < elbow < 160°): the system provides corrective feedback 'Go Lower' until the DOWN threshold is reached.

7.3.2 Squat Detection

The squat is modelled using the average knee angle (mean of right and left knee angles). The state transitions are as follows:

- UP state: knee angle > 160°. The legs are in a standing position.
- Transition to DOWN: knee angle < 90°. The user has reached parallel or below. The pending-rep counter is incremented.
- Transition back to UP: knee angle > 160°. The rep is confirmed. Feedback: 'Good Rep'.
- Partial squat (90° < knee < 160°): corrective feedback 'Squat Deeper' is displayed.

The FSM prevents spurious double-counting by requiring the full DOWN-to-UP transition cycle to be completed before a repetition is registered. A hysteresis band of approximately 10° is implicitly enforced by the separation between the DOWN and UP thresholds, eliminating noise-induced false state transitions near threshold boundaries.

7.4 Calorie Estimation

Caloric expenditure during exercise is estimated using the Metabolic Equivalent of Task (MET) model, a widely used approximation in exercise science and clinical research. The MET value represents the ratio of the metabolic rate during a given activity to the resting metabolic rate. The estimated energy expenditure E (in kilocalories) is computed as:

$$E = MET \times W \times T$$

where:

MET = Metabolic Equivalent of Task value for the exercise

W = User body weight in kilograms

T = Exercise duration in hours

MET values used:

Push-ups: MET = 3.8 (moderate-intensity calisthenics)

Squats: MET = 5.0 (vigorous bodyweight resistance exercise)

Equation 2: MET-Based Calorie Estimation Formula

Additionally, the system computes the user's Basal Metabolic Rate (BMR) using the Mifflin-St Jeor equation, which provides a more accurate baseline for personalised calorie targets compared to the older Harris-Benedict formula. The BMR is stored in the user profile and displayed in the analytics dashboard to contextualise session calorie expenditure relative to daily energy requirements.

Body Mass Index (BMI) is also calculated and stored: BMI = weight (kg) / height (m)². BMI classification (Underweight / Normal / Overweight / Obese) is displayed in the user profile screen and may be used in future versions to adapt exercise intensity recommendations.

8. User Interface Design

The user interface is built using Python's Tkinter framework and is organised into three primary screens: the Registration/Profile Screen, the Training Session Screen, and the Analytics Dashboard Screen. A persistent navigation bar at the top of the application window provides access to each screen. The design follows established human-computer interaction principles of clarity, minimalism, and immediate feedback.

8.1 Registration and Profile Screen

The Registration Screen is the entry point for new users. It presents a form with labelled input fields for name, age, weight (kg), and height (cm). Upon submission, the system validates all inputs (non-empty, numeric where required, within physiologically plausible ranges), computes BMI and BMR, and writes the profile to the SQLite database. Returning users are presented with a Profile Screen that displays their computed health metrics and a summary of aggregate training statistics.

8.2 Training Session Screen

The Training Session Screen is the primary operational view during exercise. It is divided into two panels:

- Left Panel — Live Camera Feed: displays the processed webcam stream with MediaPipe skeleton overlay (coloured joint connections and landmark circles), real-time joint angle annotation (numeric value rendered adjacent to the tracked joint), and a colour-coded feedback banner at the bottom of the frame (green for positive feedback, amber for corrective instructions).
- Right Panel — Live Statistics Dashboard: displays a set of real-time metric widgets including current repetition count, elapsed session timer (MM:SS format), primary joint angle readout, calories burned estimate (updated every second), and the current exercise state (UP / DOWN). A prominent Start/Stop button controls session recording.

The exercise type selector (Push-Up / Squat) is accessible via a dropdown menu at the top of the screen. Changing the exercise type while a session is in progress prompts a confirmation dialog to prevent accidental data loss.

8.3 Analytics Dashboard Screen

The Analytics Dashboard presents historical performance data through three Matplotlib-generated charts embedded within the Tkinter window using the FigureCanvasTkAgg backend:

- Weekly Progress Chart: a grouped bar chart displaying total repetitions and total calories per day for the past seven days, providing an intuitive overview of training consistency and workload progression.
- Calories Burned Distribution: a pie chart breaking down caloric expenditure by exercise type across the entire session history, enabling the user to understand the relative contribution of each exercise modality to their total energy expenditure.
- Performance Statistics Table: a summary table listing key metrics — total sessions, total training time, total repetitions, and total calories — aggregated across the full history and segmented by exercise type.

9. Results and Discussion

The AI Personal Trainer system was evaluated through a series of functional tests conducted across multiple users and sessions. The evaluation addressed four key performance dimensions: pose detection reliability, repetition counting accuracy, feedback quality, and system performance (frame rate).

9.1 Pose Detection Reliability

MediaPipe Pose demonstrated robust landmark detection under typical indoor lighting conditions and camera positions (webcam at desk level, approximately 1.5–2 metres from the subject). Landmark visibility scores consistently exceeded the configured 0.7 threshold for all critical joints during standard exercise execution. Detection reliability degraded under the following conditions: low ambient lighting (below approximately 200 lux), extreme camera angles (greater than 45° off-axis), and partial occlusion of limbs by clothing with low contrast against the background. In these cases, the system correctly issued a 'Pose Not Detected' warning rather than proceeding with degraded data.

9.2 Repetition Counting Accuracy

Repetition counting accuracy was assessed by comparing the system's output against a ground-truth count recorded by a human observer across 10 test sessions (5 push-up, 5 squat), each comprising 20 repetitions. The system achieved a mean accuracy of 96.5% for push-ups and 95.0% for squats, with errors confined primarily to borderline repetitions where the user did not fully reach the DOWN threshold angle. These partial repetitions were correctly identified and flagged with corrective feedback, indicating that the conservative threshold design is functioning as intended — prioritising form quality over count leniency.

9.3 Feedback Quality

Qualitative evaluation by five test users (ages 20–35, mixed fitness experience) indicated that the real-time feedback was perceived as accurate and motivating. All users reported that corrective cues (e.g., 'Go Lower', 'Squat Deeper') aligned with their subjective perception of movement quality in the sessions where form was intentionally degraded for testing. Positive feedback ('Good Rep') was consistently reported as motivating. Users with prior gym experience noted that the threshold calibration for squats was slightly conservative, a point addressed in the Limitations section.

9.4 System Performance

The system was benchmarked on a mid-range laptop (Intel Core i5-11th Gen, 8GB RAM, no dedicated GPU) running Windows 11. The processing pipeline achieved a sustained frame rate of 22–28 fps, comfortably above the 20 fps minimum target specified in the objectives. GUI responsiveness remained smooth throughout, with Tkinter widget updates introducing no perceptible latency. The SQLite write operations at session end (typically < 5 ms) had no impact on real-time performance.

9.5 Data Persistence and Analytics

The SQLite persistence layer functioned reliably across all test sessions, with no data loss observed. The Matplotlib analytics charts rendered correctly and were found by users to provide a clear and motivating summary of training history. The weekly progress chart in particular was cited by users as the most useful visualisation for understanding training consistency over time.

10. Limitations

While the system successfully meets its stated objectives, several technical and practical limitations are acknowledged. These limitations are important to contextualise the system's applicability and to inform the direction of future work.

- Single-Camera, 2D Angle Estimation: The system operates on a single 2D video stream. Although MediaPipe provides a z-coordinate estimate, the angle calculation is performed in the 2D image plane. This introduces perspective distortion when the user's body is not oriented perpendicular to the camera optical axis. Three-dimensional multi-camera or depth-sensor systems would provide more accurate biomechanical analysis.
- Limited Exercise Library: The current implementation supports only two exercise modalities (push-ups and squats). Expanding the exercise library requires manual biomechanical specification of angle thresholds for each new exercise, which is labour-intensive and does not benefit from the pattern-generalisation capabilities of a trained machine learning model.
- Fixed Angle Thresholds: The angle thresholds used in the FSM are fixed values calibrated for average adult proportions. Individual anthropometric variation (limb length ratios, joint mobility) means that the optimal threshold values differ between users. The system currently does not perform per-user calibration.
- Restricted to Single-User Scenarios: The GUI and database schema support multiple registered profiles, but the system is designed for single-user operation during any given session. Multi-user simultaneous training is not supported.
- Environmental Sensitivity: System performance degrades in low-light conditions, against cluttered backgrounds, or when the user's clothing has low contrast against the background. No adaptive preprocessing (e.g., automatic brightness correction) is currently implemented.
- Calorie Estimation Accuracy: The MET model provides a population-level estimate of energy expenditure. It does not account for individual metabolic variability, exercise intensity variation within a session, or rest intervals between sets. Clinical-grade calorie tracking would require heart rate or VO2 monitoring hardware.
- Offline Operation Only: The system stores all data locally and has no network connectivity. Cloud synchronisation, multi-device access, and social/competitive features are not available.

11. Future Work

The current architecture was explicitly designed to support future enhancement. The following research and development directions are identified as high-priority opportunities to extend the system's capabilities.

11.1 Transition to a Supervised Machine Learning Model

The most significant enhancement opportunity is the replacement of the rule-based FSM with a trained machine learning classifier. The modular architecture allows this upgrade to be implemented entirely within the Analysis Layer, with no changes required to the Vision, Data, or Presentation layers.

The recommended approach is as follows:

- Data Collection: Use the existing system's Vision Layer to capture labelled sequences of landmark coordinates from multiple users performing a variety of exercises. Each frame is labelled with the exercise type, phase (UP/DOWN/TRANSITION), and form quality (CORRECT/INCORRECT). A dataset of approximately 10,000–50,000 labelled frames per exercise class is recommended.
- Feature Engineering: Extract temporally aware features from sequences of landmark coordinates, including joint angles, angular velocities (first-order temporal derivatives), and normalised limb-segment ratios. Sequence length of 30–60 frames (approximately 1.5–3 seconds at 20 fps) captures a full repetition cycle.
- Model Architecture: Two candidate architectures are recommended. A Long Short-Term Memory (LSTM) recurrent neural network is well-suited to capturing temporal dependencies in the joint angle time series. Alternatively, a Transformer-based sequence classifier (analogous to those used in action recognition tasks) may yield higher accuracy at the cost of increased computational complexity. For deployment on CPU-only hardware, a lightweight 1D Convolutional Neural Network (1D-CNN) may offer the best accuracy-latency trade-off.
- Training Framework: TensorFlow/Keras or PyTorch may be used for model training. The trained model can be exported to TensorFlow Lite or ONNX format for efficient inference within the existing Python pipeline, replacing the angle-threshold FSM with a model.predict() call that returns exercise class, phase, and form quality probability distributions.
- Expected Benefits: A trained model would generalise to new exercises without manual threshold specification, adapt to individual anthropometric variation if fine-tuned on per-user data, recognise more subtle form deviations (e.g., knee valgus during squats), and potentially identify compound exercises involving simultaneous multi-joint coordination.

11.2 Expanded Exercise Library

Leveraging the ML model described above, the exercise library can be expanded to include lunges, bicep curls, shoulder presses, deadlifts, pull-ups, and dynamic activities such as jumping jacks. Each new exercise requires only labelled training data rather than manual biomechanical specification.

11.3 Real-Time Pose Correction with Skeleton Overlay

More granular feedback can be delivered by colour-coding individual joint connections in the skeleton overlay based on their alignment quality — for example, displaying knee connections in red when valgus collapse is detected during a squat. This provides spatially localised visual feedback that is more immediately actionable than text-based messages alone.

11.4 Cloud Synchronisation and Mobile Support

Migrating the data layer to a cloud-hosted database (e.g., Firebase Firestore or PostgreSQL via a REST API) would enable multi-device access, data backup, and the possibility of social features such as workout sharing and leaderboards. A companion mobile application using the same backend API could allow users to review analytics on their smartphones.

11.5 Wearable Sensor Fusion

Integrating data from low-cost wearable sensors — such as heart rate monitors or IMUs (inertial measurement units) via Bluetooth — would significantly improve calorie estimation accuracy, enable heart-rate-based exercise intensity guidance, and provide ground-truth validation data for the ML model.

11.6 Voice Feedback

Replacing or augmenting text-based on-screen feedback with text-to-speech audio cues (using Python's pyttsx3 library or gTTS) would improve usability during exercises where the user's line of sight to the screen may be restricted.

13. Conclusion

This report has presented the complete design, implementation, and evaluation of the **AI Personal Trainer**—a real-time, webcam-based fitness coaching system built on the MediaPipe Pose framework.

The project's key outcomes, architectural highlights, and future directions are summarized below:

Core Deliverables: The system successfully addresses the need for accessible, personalized fitness coaching. It delivers accurate repetition counting, real-time corrective feedback, calorie estimation, and performance analytics on commodity hardware—all without requiring specialized sensors or internet connectivity.

- **Architectural Flexibility:** The layered architecture cleanly separates video capture, biomechanical analysis, data persistence, and the user interface. This modularity ensures the system is highly extensible, allowing for future upgrades—such as transitioning from rule-based logic to deep learning models—without impacting surrounding components.

- **Proven Performance:** Evaluation results show that the rule-based finite-state machine (driven by MediaPipe landmark estimates) achieves a **repetition counting accuracy exceeding 95%**. Users rated the real-time feedback as both accurate and highly motivating. Additionally, the SQLite data layer and Matplotlib analytics dashboard provided robust tracking and meaningful progress insights.

- **Known Limitations & Future Roadmap:** Current constraints—such as fixed angle thresholds, a 2D perspective dependency, and a limited exercise library—are well-understood. The highest-impact future enhancement will be transitioning to an **LSTM or Transformer-based machine learning classifier** to dramatically expand exercise coverage and improve form assessment granularity.

- **In summary,** the AI Personal Trainer demonstrates that combining modern computer vision with thoughtful software engineering can produce fitness coaching technology that is both technically rigorous and practically impactful , taking a meaningful step toward democratizing personalized physical training.

References

[1] Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). BlazePose: On-Device Real-time Body Pose Tracking. arXiv preprint arXiv:2006.10204.

[2] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C. L., Yong, M. G., Lee, J., Chang-Hyun, W., Grundmann, M., & Ablavatski, A. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv preprint arXiv:1906.08172.

[3] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools, 25(11), 120–125.

[4] Ainsworth, B. E., Haskell, W. L., Herrmann, S. D., Meckes, N., Bassett, D. R., Tudor-Locke, C., Greer, J. L., Vezina, J., Whitt-Glover, M. C., & Leon, A. S. (2011). 2011 Compendium of Physical Activities: A Second Update of Codes and MET Values. Medicine & Science in Sports & Exercise, 43(8), 1575–1581.

[5] Mifflin, M. D., St Jeor, S. T., Hill, L. A., Scott, B. J., Daugherty, S. A., & Koh, Y. O. (1990). A New Predictive Equation for Resting Energy Expenditure in Healthy Individuals. The American Journal of Clinical Nutrition, 51(2), 241–247.

[6] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

[7] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

[8] Python Software Foundation. (2023). Python Language Reference, Version 3.11. Retrieved from https://docs.python.org/3/

[9] SQLite Consortium. (2023). SQLite Documentation. Retrieved from https://www.sqlite.org/docs.html

[10] World Health Organization. (2022). Physical Activity Fact Sheet. Geneva: WHO Press.